High-Demand vs. Low-Demand NER on Small Language Models

Claire Wang

1 Introduction

Large language models (LLMs) have achieved state-of-the-art results on many natural language processing tasks, but named entity recognition (NER) presents various challenges. For example, an LLM may fail to return data in the correct format. Previous NER models often use token-level sequence labeling, which is difficult for LLMs to do. For small language models (SLMs) that have been trained on less data and have fewer resources at inference time, these challenges are exacerbated.

Previous research on non-NER tasks has shown that the performance of small language models can be constrained by task demands auxiliary to the actual capability being evaluated. My project applies this idea to NER by proposing a "high-demand" and "low-demand" evaluation method / data format, and fine-tuning the model using the same format.

Since I am investigating a question, I chose the **Research** track for my project. The independent variable of demand level (approximated through data format) was used for data processing, training, and evaluation, but since the bulk of my work went into developing and executing the training methodology, I chose **Training** as my focus.

2 Related Work

My work was primarily inspired by Hu & Frank (2024), who considered the psychological concept of task demands (auxiliary challenges) and hypothesized that when assessing the capabilities of a small language model, of the evaluation method may negatively affect the model's performance. They found that performance was indeed better when demands were reduced for the tasks of "analogical reasoning, reflective reasoning, word prediction, and grammaticality judgments".

Wang et al. (2025) used a language model to achieve NER performance comparable to fully supervised baselines , but they worked with GPT-3, which at 175B parameters is much larger than any of the models I use.

3 Data

The dataset I used was Tweebank-NER, which was created by adding NER annotations to Tweebank V2 (a collection of English tweets) (Jiang et al., 2022). The annotators were recruited through Amazon Mechanical Turk. I accessed it through OpenNER, a standardized collection of NER datasets published on HuggingFace (Palen-Michel et al., 2025).

The dataset has 3,550 tweets total: 1,639 in the train set, 710 in the dev set, and 1,201 in the test set. I kept these splits but used the dev split as a second set of training data (see the Training section for an explanation).

The data is pre-tokenized and provides one label per token in BIO format. The entity types are person (PER), location (LOC), organization (ORG), and miscellaneous (MISC). User mentions, hashtags, and URLs were not annotated, even if they referred to an entity. In total, there are 2154 mentions: 777 PER, 317 LOC, 541 ORG, and 519 MISC.

The only preprocessing I did was to remove "RT [username] :" from the beginning of any tweets where it appeared, since this was only an indication that the user retweeted the tweet and irrelevant to the substance of the tweet.

4 Methods

4.1 Changing Demand Level

I trained each model twice (I started with the same base model, but the runs were completely separate). The first training process was the "high-demand" one, and the second was the "low-demand" one. The primary difference between them was the data format, which looked as follows:

***High Demand*:**

Prompt: "The EU tackles climate change " by ignoring the building of coal fired power stations in Germany and puts VAT on solar p . . .

Expected Response: " The «ORG: EU » tackles climate change " by ignoring the building of coal fired power stations in «LOC: Germany » and puts «MISC: VAT » on solar p . . .

***Low Demand*:**

Prompt: Sentence: "The EU tackles climate change " by ignoring the building of coal fired power stations in Germany and puts VAT on solar p . . . ". The type of "Germany" is:

Expected Response: LOC

The high-demand format provides a tweet as the user prompt and expects the model to return the tweet exactly as entered, with entities added inline. This format was inspired by Wang et al. (2025), but their start and end tokens were @@ and ##. Since tweets contain many @s and #s as part of the data, I changed these to « and ». The idea of adding the entity type before the mention was taken from Palen et al. (2025).

The low-demand format provides a tweet but also provides a span from the tweet and asks for the entity type (or O if the span does not represent an entity). This only requires the model to generate one token and thus leaves much less room for error in the model response.

An issue with this format is that it is not feasible to test all possible O spans, (i.e., spans that do not contain a mention). I chose to use the set of false positives that came from evaluating the high-demand version of the model on the test set. Since these were the O spans that the model incorrectly predicted as mentions, I believed these would be the O spans that would be most difficult for the low-demand version of the model to correctly predict and allow for a reasonable comparison between the two model versions.

For each format, there was also a system prompt describing the task (see Appendix).

4.2 Training Process

For the high-demand level, I fine-tuned the model on the train dataset and then evaluated it on a second train dataset (the dev split from the original dataset).

For the low-demand level, I fine-tuned the model on the second train dataset, using all correct mentions and all false positives from the high-demand model's evaluation. I then evaluated it on the test set to compare results with the high-demand model.

4.3 Training Setup

I used the base versions of four models: Llama 3.1 8B, Mistral v0.3 7B, Qwen 3 4B, and Gemma 4 E4B.

The fine-tuning procedure was QLoRA, performed with Unsloth and Hugging Face's Supervised Fine-Tuning Trainer. I ran one epoch for each round and changed the learning rate to 2e-5 but left all other hyperparameters as their default values.

All experiments were run on Google Colab, with code adapted from Unsloth's Colab notebooks.[1]

4.4 Evaluation

I evaluated the model's performance by comparing the predicted mentions to the true mentions and calculating precision, recall, and F1-score. For mentions of the wrong type, I awarded "partial credit" by only adding 0.5 each to the false negative and false positive counts (traditional NER evaluation would add 1 to both counts). There was no such tolerance for partially correct spans.

[1] https://github.com/unslothai/notebooks

Model	Llama	Mistral	Qwen
HD Train 2	29.75	68.10	64.73
HD Test	34.84	65.59	62.68
LD Test	67.08	75.10	73.41

Table 1: F1 scores for high-demand (HD) model on train set 2, HD model on test set, and low-demand (LD) model on test set.

Class	PER	LOC	ORG	MISC
Llama HD	52.53	37.79	24.51	15.58
Llama LD	75.75	74.32	68.10	47.30
Mistral HD	81.71	74.07	65.24	37.80
Mistral LD	92.59	73.44	62.65	57.24
Qwen HD	78.76	71.50	59.73	34.99
Qwen LD	80.51	79.12	70.16	62.13

Table 2: Per-class F1 scores for high-demand (HD) and low-demand (LD) models on test set.

For the high-demand data format, malformed mentions were ignored (e.g. if there was an opening tag but no closing tag).

Because the primary focus of my work was comparing performance between the high-demand and low-demand processes, I did not have a traditional baseline.

5 Results

F1 scores are reported in Tables 1 and 2. Results for Gemma are not reported because it did not produce usable output during the high-demand evaluation. Instead of annotating any entities, it returned the input tweet with one word changed.

6 Analysis & Discussion

As expected, the low-demand model outperformed the high-demand model. Generation was also significantly faster: the test set eval took close to an hour for the high-demand model and less than 15 minutes for the low-demand model.

However, larger size did not correlate with better performance. The high-demand Llama model, despite being the largest at 8B parameters, performed significantly worse than any of the other models. For a more rigorous comparison, future work would use different sizes of model from the same model family.

Looking at Table 2, MISC was the worst-performing category, which aligns with previous results on NER for both LLMs and other model types. The lower-demand scores were higher for every category and model except LOC and ORG for Mistral. Scores dramatically improved for Llama especially.

Directions for future work could include trying different data (from other sources or in other languages), using a more diverse range of model sizes, tuning hyperparameters, and looking at token probabilities (something Hu & Frank did and I had originally hoped to do) or more modern mechanistic interpretability oriented strategies.

7 Conclusion

We have shown that a lower-demand version of the NER task significantly improves performance for several small language models from different families. More research is required before any definitive conclusions can be drawn, but this project serves as a promising start for exploration.

References

Jennifer Hu and Michael C Frank. 2024. Auxiliary task demands mask the capabilities of smaller language models. *arXiv preprint arXiv:2404.02418*.

Hang Jiang, Yining Hua, Doug Beeferman, and Deb Roy. 2022. Annotating the Tweebank corpus on named entity recognition and building NLP models for social media analysis. In *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, pages 7199–7208.

Chester Palen-Michel, Maxwell Pickering, Maya Kruse, Jonne Sälevä, and Constantine Lignos. 2025. Open-ner 1.0: Standardized open-access named entity recognition datasets in 50+ languages. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 33637–33662.

Shuhe Wang, Xiaofei Sun, Xiaoya Li, Rongbin Ouyang, Fei Wu, Tianwei Zhang, Jiwei Li, Guoyin Wang, and Chen Guo. 2025. Gpt-ner: Named entity recognition via large language models. In *Findings of the association for computational linguistics: NAACL 2025*, pages 4257–4275.

A System Prompts

High-Demand: You are an expert annotator. Find named entities in the input and enclose them with « and ». Add the entity type (PER:, LOC:, ORG:, or MISC:) after each «. Return only the input sentence with the added labels.

Low-Demand: You are an expert annotator tasked with labeling a word/phrase from a sentence. If the word/phrase is a named entity, return its type (PER, LOC, ORG, or MISC). If the word/phrase is not a named entity, return O.